

Predicting Major League Baseball Future Wins Above Replacement

Jackson Neeley

ITCS-3156-001

Contents

1. Introduction	3
1.1 Problem Statement	4
1.2 Motivations	4
1.3 Challenges	4
1.4 Summary of Approach	5
2. Data	6
2.1 Analysis	6
2.2 Preprocessing	6
3. Ridge Regression	7
3.1 Understanding Approach With Ridge Regression	7
3.2 Ridge Regression Data	8
3.3 Issues Faced With Ridge Regression Model	10
4. Random Forest	13
4.1 Understanding Approach With Random Forest	13
4.2 Random Forest Data	14
4.3 Issues Faced With Random Forest	15
5. Conclusion	16
5.1 Closing Thoughts	16
5.2 Challenges Faced	17
5.3 What I Learned	19
References	20
Acknowledgments	21
Source Code	21

1. Introduction

In its simplest terms, the game of baseball is the act of one throwing to another hitting, and from the outlook of those outside of the game, there isn't much more. Alternatively, the game can be seen as an accumulation of numbers and probabilities, numbers that can tell a story of specific actions, the future of the sport, and even the entire career of players. In the world of baseball these numbers are considered Sabermetrics (originally SABRmetrics, after the Society for American Baseball Research), coined by historian, statistician, and baseball writer Bill James in 1980. Before these advanced metrics, Major League teams had access to basic probabilities such as averages (AVG), earned runs average (ERA), and counting stats like Runs Batted In (RBI), hits (H), and wins (W) and losses (L). Teams used these basic counting statistics to value players and determine their abilities.

In Sabermetrics, we take these basic values and contort them in such a way that we can determine a player's true value based on multiple different variables. A great example would be Adjusted On-base Plus Slugging Plus (OPS+). OPS takes the On-base percentage of a player ($\frac{H+BB+HBP}{PA}$) and combines it with a players slugging percentage ($\frac{TB}{AB}$). Finally the plus (+) part of the metric adjusts each AB based on park factors and averages the number to 100, where a player with an OPS+ of less than 100 is seen as below average and higher than 100 to be above average. What these metrics allow us to do is to see a player's value that isn't just based on numbers alone but instead include different factors. Sabermetrics aren't just limited to factors, with the advancements of technology, teams are able to receive tracking data such as a players sprint speed, reaction time, and even numbers like a pitches rotations per minute and bat swing speed. These metrics alone can determine how well a player is able to be successful throughout the stages of their career.

1.1 Problem Statement

In accordance with the ever growing metrics in Major League Baseball, I have decided to construct an algorithm that will help predict a player's future Wins Above Replacement (WAR). WAR is an algorithm that can tell how much a player's value differs from a replacement player. A replacement player is a hypothetical player who represents the minimum talent needed for a Major League team. The best way to look at this replacement player is that they are simply a Minor League player with the talent able to replace someone at the Major League level. A player whose WAR is less than 0.0 could be seen as a Minor League player and one who has a WAR greater than 0.0 has the talent to at least survive in the Majors. To keep this simple I have chosen to only do batting WAR, in such that no types of pitching or fielding will be used.

1.2 Motivations

My motivation for this topic is very simple: I love baseball. As someone who has grown up taking in baseball every single day of my life for almost 20 full years, why wouldn't I choose a topic I am so passionate about. Not only have I fallen in love with the game itself, but I have fallen in love with the statistical aspect of the game. Baseball is an ever growing sport that relies on the future of technology to further expand its outreach.

1.3 Challenges

The biggest challenge I have faced was issues with pybaseball and its Fangraphs import. Fangraphs provides a full database on players since 2015 that contains not only counting stats, but all of a player's rate stats and Sabermetrics. More importantly I was hoping to import Statcast models with Fangraphs. Statcast is Major League Baseball's tracking technology that they have used since 2015. These high speed cameras are able to track a player's sprint speed, swing speed, ball movement, and a lot of other points of data that simple statistics can't track. With this comes

the issue, many times throughout the year Fangraphs likes to update their website frequently.

Most of these changes like to come near the start of the season where they are updating metrics and fixing adjustments. The Fangraphs import from pybaseball relies on web scraping and when the website is seeing a structural update the import breaks due to change in the HTML.

Unfortunately I ran into this issue while working on this project, and had to resort to a different approach that was less in depth.

1.4 Summary of Approach

My general approach for this project is to use two different algorithms to attack different problems, then to blend them together. First I want to create such a model that is able to predict the general populus of players. For this I have chosen to use a ridge regression model that should be able to figure out the average players performance. With that I need to create another model that will be able to closely predict outliers such as star players. Using a random forest where I boost these outliers should help with their properly predicted WAR. Finally I will try to blend both these models together in order to find a sufficient model to showcase.

2. Data

For this project I have to move over to using the public Lahman database, which has tracked all discrete counting stats since the late 1800's. When trying to constraint the data I decided to land in 2015 where baseball is generally known to enter the Statcast era. This gives me a 10 year window to experiment with modern baseball.

2.1 Analysis

Analyzing the data I realized that Lahman doesn't track the most important stat I need, WAR. I also realized that no rate stats were being tracked as well, so I had to go in and create my own rate stats and my own variation of WAR. To be clear, WAR is a very convoluted stat that takes in multiple factors that I cannot get my hands on, so I had to create my own replacement variables and factors. Overall I thought my approach to this problem was acceptable and found my own WAR to be decently similar to other WAR algorithms.

2.2 Preprocessing

The first step in my preprocessing was to set my features I wanted to use, which were 14 stats that I believed had the strongest correlation to WAR. Next I set my target as "future_WAR" or more specifically the current seasons predicted WAR. I then needed to engineer the values of "future_WAR" which I did by creating a weighted war variable that would weight the previous three years of WAR based on recency. I also created a trend for the WAR and a variable to hold the squared age of a player. Finally clean up the data, split it up, then return the full dataframe.

3. Ridge Regression

To get a base starting point for this problem I have created a fairly simple ridge regression model. As stated this model is to focus more on figuring out the average players performance than to figure out the “elites”. For my data I want to focus on staying less than 2 WAR of error from their actual WAR and I will consider any values above a 3 WAR difference to be considered an outlier. With that out of the way, here are my general error values for the ridge regression algorithm.

```
Ridge Regression
MAE:  1.243
RMSE: 1.683
R2:  0.478
```

3.1 Understanding Approach With Ridge Regression

Before we can determine the prediction values, it is important to look at the approach and the top feature coefficients and how they affect the model. Creating a model of the top 10 features allows us to see what affects the model the most.

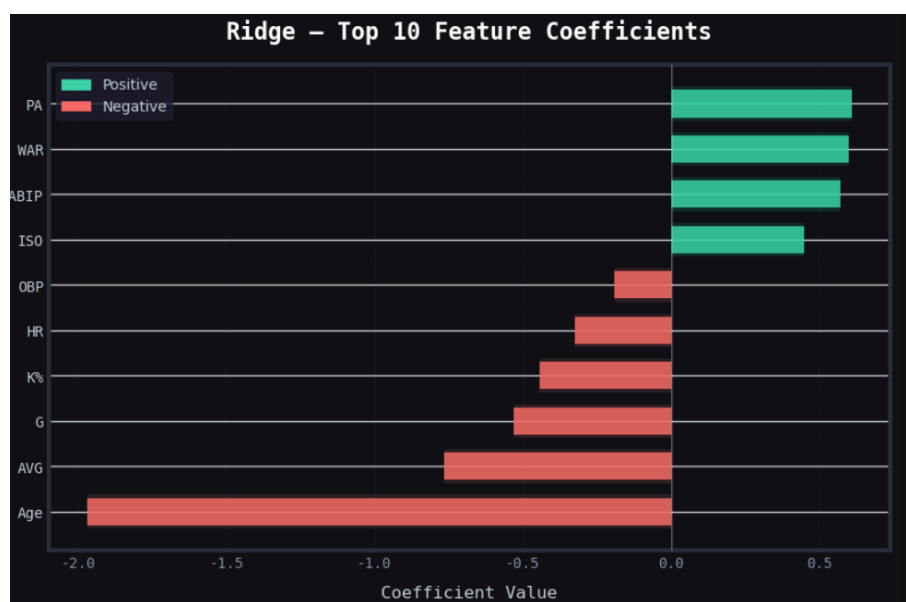


Figure 1.1

A negative coefficient value negatively affects a WAR of a player, this is most obvious when looking at the age of a player. Based on the model a player whose age exceeds past their prime (the peak of their career) sees a drop of almost 2 WAR a season. More interesting though is I would like to discuss the positive values, for starters PA (plate appearances) suggest that the more playing time a player gets, the more their WAR value goes up. This makes the most sense, considering that the best players will get more playing time causing more plate appearances and ultimately growing their WAR. Speaking of WAR, the model suggests that a player's previous WAR greatly affects their next season's WAR. The best way to look at this is very simple, a player's previous season's production will show us an accurate representation of their next season production. Someone on a decline is more likely to continue declining and someone who is rising will likely rise the following year.

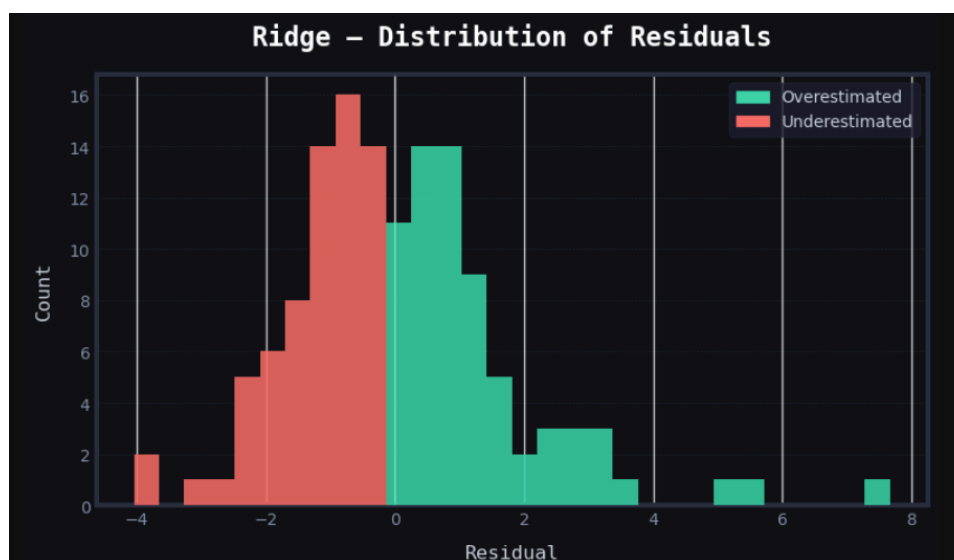


Figure 1.2

3.2 Ridge Regression Data

After running the regression model, we can view the predictions of WAR vs a players actual WAR in 2024 in Figure 1.1.

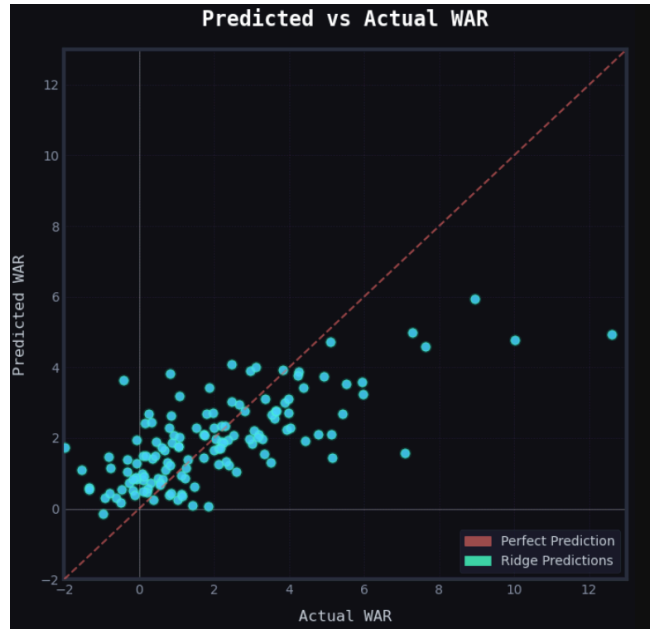


Figure 1.3

The linear red dotted line that runs through the chart represents the perfect prediction, for example a prediction WAR of 2 should have an actual WAR of 2. The scattered data on the model shows its ridge predictions. To get a better look at the difference between the ridge predictions and the perfect prediction I have generated heat maps detailing the distance of data points from their desired points.

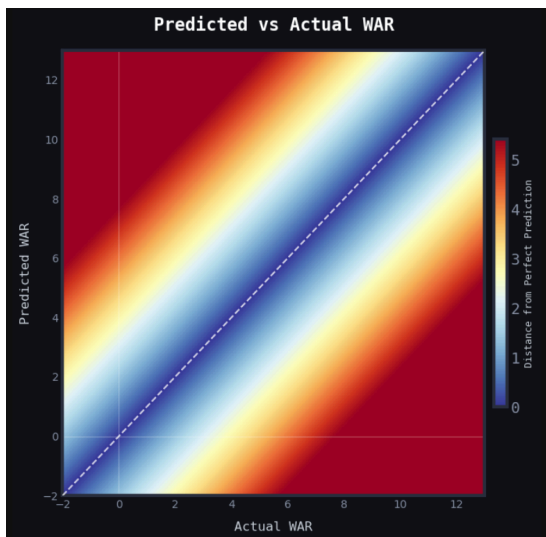


Figure 1.4

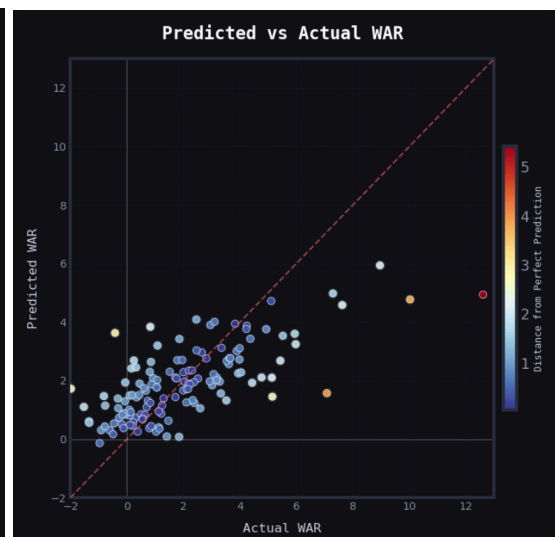


Figure 1.5

Figure 1.4 shows the heat map and where the most desirable locations are, at least 1 WAR between the prediction and actual outcome. Looking at Figure 1.5 shows where the data points are in comparison to the heat map. From the looks of the map it can be seen that the model has an easier time with predicting average players (players between 0-3 WAR) versus who underperform or overperform. The model seems to overpredict players who are seen as less than league replacement and severely underpredict players who showcase at an all star to a MVP caliber season.

3.3 Issues Faced With Ridge Regression Model

As discussed just prior, the ridge regression model likes to shift the extremes to the mean. This is done by the model compressing the data points to the center of distribution by shrinking coefficients. Ridge regression is more trying to get the mean, the average player, more accurately than the outliers, such as star players. Outputting the range and mean of the model shows this correlation perfectly.

```
Actual WAR range: -1.9814782608695656 12.605768115942029
Predicted WAR range: -0.13540902400885413 5.935683262086453
Mean Actual WAR: 1.9764446591519054
Mean Predicted WAR: 1.9124625328559155
```

The predicted range displayed is much smaller, by more than half of the actual range, but the means are separated by less than a tenth of WAR. To better visualize this phenomenon I have isolated the 10 outliers that have an error of 3 more WAR.

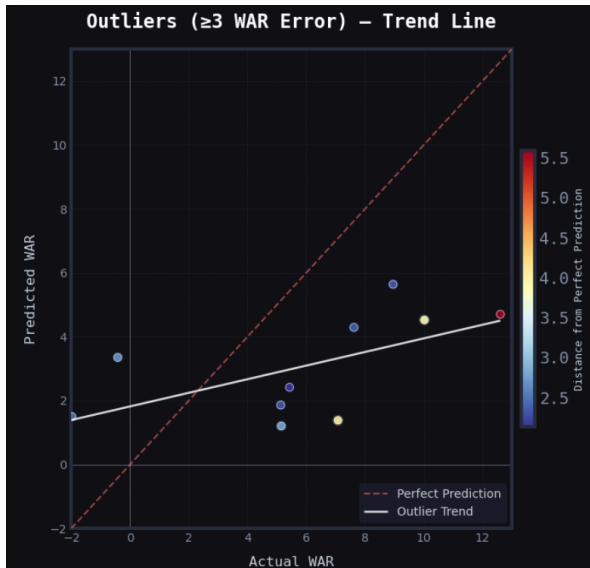


Figure 1.6

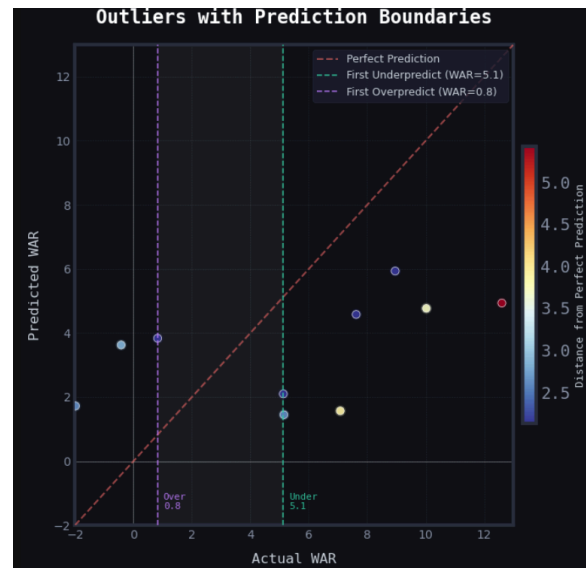


Figure 1.7

As thought, the model likes to overpredict players with less than 2 total WAR, then jumping to underpredicting players after intersecting with the perfect prediction line as showcased in Figure 1.6. Figure 1.7 displays the regions separating the over and underpredictions while also highlighting the regression model is most accurate at predicting WAR between 0.8 and 5.1 actual WAR. Looking at the outline trend line in Figure 1.6, we can very obviously determine that the slope is much less than one, more accurately the slope is 0.176. This further proves the point that the ridge regression model refuses to predict extremes in either direction, known as bias-variance tradeoff. The model is more focused on achieving a lower variance around then mean, and in doing so it accepts higher biases.

Although being able to predict an average player is an awesome tool, baseball is not just a bunch of average Joes. There exists such star players that need to be predicted correctly, especially when the goal of these predictions are to find players who have a chance of breaking out and becoming stars. Ridge regression is good at predicting in between the biases but is designed to not overfit, penalizing larger coefficients just so it can more accurately predict the

mean. Moving forward I'd look to move on to an algorithm that can support not only the average players but also support bad and great players.

4. Random Forest

Now towards improving on my previous attempts I have developed a random forests model. For this model I focus more on improving the “elites” (war outside of the ridge model predictions) than focusing on the general mean of a player.

```
Random Forest (2-stage)
MAE:  1.320
RMSE: 1.771
R2:  0.422
```

4.1 Understanding Approach With Random Forest

This approach was less than about understanding the residuals and more of finding an algorithm that more accurately predicted the “elites”. Just like the last model I have set up the same graph.

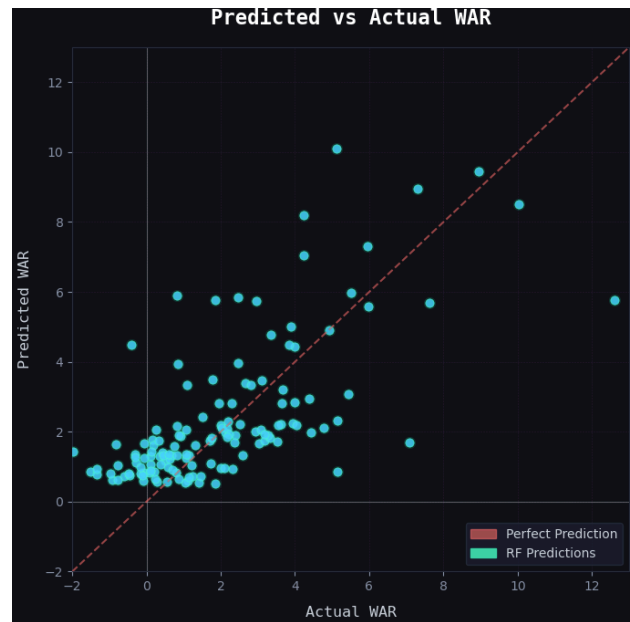


Figure 2.1

As we can see the higher values are more accurately predicted than shown in the ridge regression model. Although with the payoff that the overall mean is much more diverse. A key issue I faced with the approach is how random forests work in the first place, with less

overfitting the values can be spread out more. Knowing this I added a boost to the “elites” setting up a general focus on these better players and raising their floors.

4.2 Random Forest Data

Following the same principle in the ridge regression model, this heat map of the data points should allow us to get a better understanding of the data.

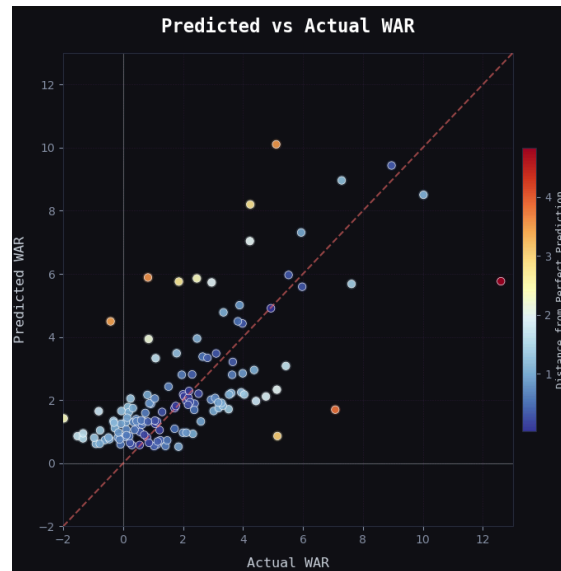


Figure 2.2

The most important thing to point out is that the general form of “elites” follow the perfect prediction line. Outside of still a couple outliers the star players are simply predicted better than in the ridge regression model.

A topic I brought up in my previous model is isolating the outliers and seeing where the algorithm first overpredicted and underpredicted data points.

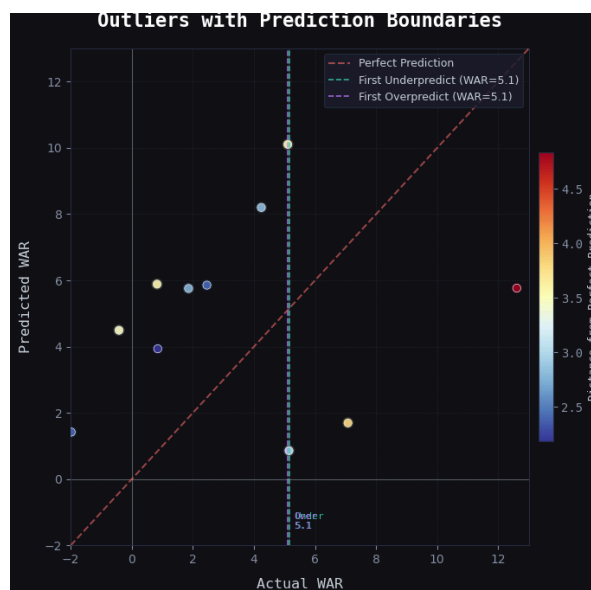


Figure 2.3

Doing the same with the random forests model, we see that the model both first overpredicts and underpredicts at the same points, 5.1 WAR. This brought up two big topics for me, the first being how greatly random forests overpredicts the average player. My assumption is that this is due to my boost to the “elites” and the model assuming these players are grouped into this category. My second take away is that the model underpredicts at the exact same data point as in the ridge regression model shown in Figure 1.7. My hypothesis for this is that they both underpredicted the same player in the same just for that player to have a surprisingly great year. I do not fault the models for this as baseball can just be as random as it can be predictable. What matters the most is the general average of the “elites” were improved upon.

4.3 Issues Faced With Random Forest

The most telling issues I had faced were figuring out how to boost the “elites” but also I had issues trying to not worry about the average. After every run I wanted to improve the mean of the model but had to hold myself to focus more on the boosting. Overall I shifted my main focus in reducing the error of the outliers to less than 2 WAR per player till it was reasonable.

5. Conclusion

5.1 Closing Thoughts

To round out my closing thoughts, I have decided to blend both models together, taking the average of ridge regression and combining it with the “elites” of the random forests model. Using the 2024 data I have showcased throughout my previous samples, here is what the final model looks like:

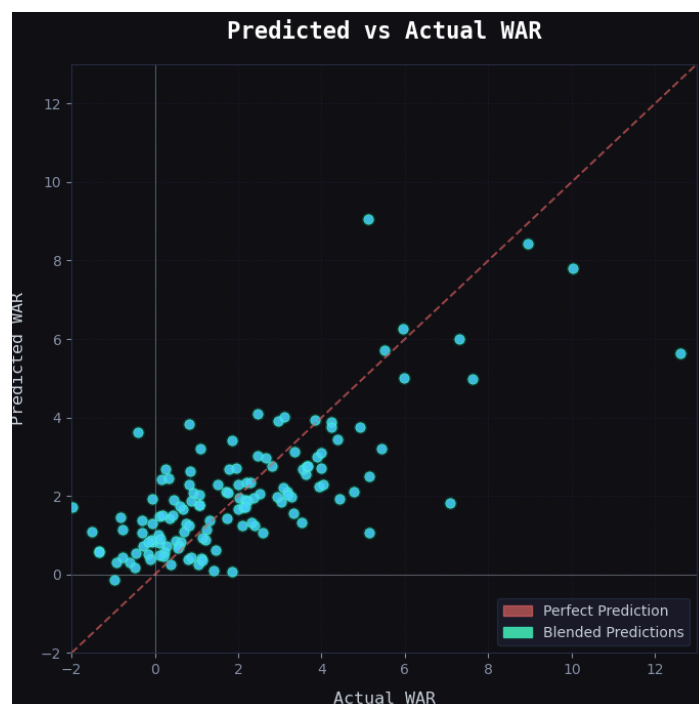


Figure 3.1

My final thought on this model is that I think it succeeds in most of the aspects. The model follows the perfect prediction line for most data points, excluding specific outliers. To get a better look at the model I have also generated the model for 2023 and 2025:

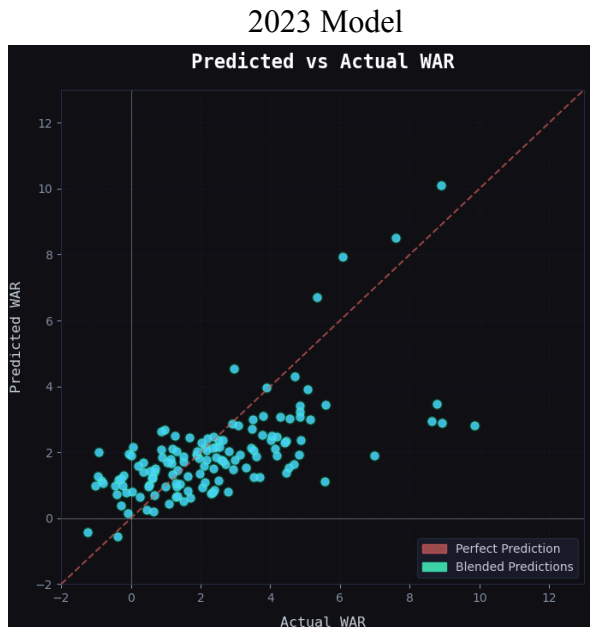


Figure 3.2

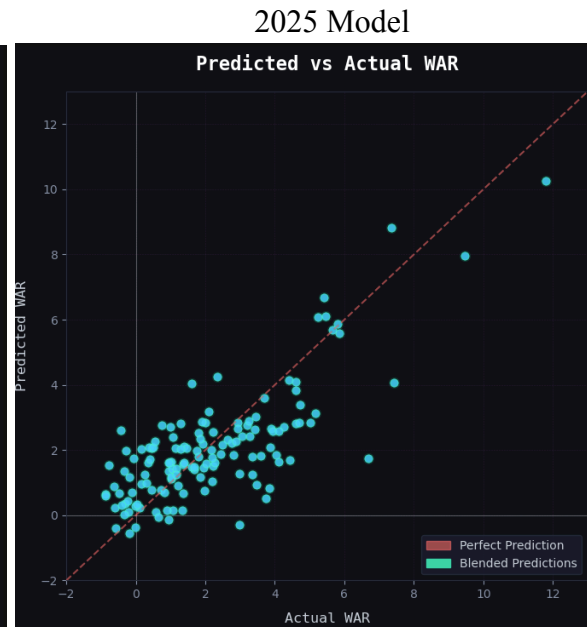


Figure 3.3

5.2 Challenges Faced

As talked about previously, one of my biggest challenges faced were the extreme outliers that couldn't be generally predicted even when switching to random forests. I have come up with a couple theories on why this is the case with the first being injuries exist. There were multiple cases of players being overpredicted by more than 4 to 5 WAR, and a good assumption would be due to said player getting injured and not being able to match their predictions based on less games played. Another issue was severe underpredictions which I have chalked up to young players still in their first three years of playing, breaking out and overperforming. This model isn't perfect and going back three years and taking only counting stats and rate stats prove that. A young star with only 2 years of experience might not be able to have a full prediction without that third year.

Another issue due to taking the previous three years is that the model underperforms in 2021, 2022, and slightly underperforms in 2023.

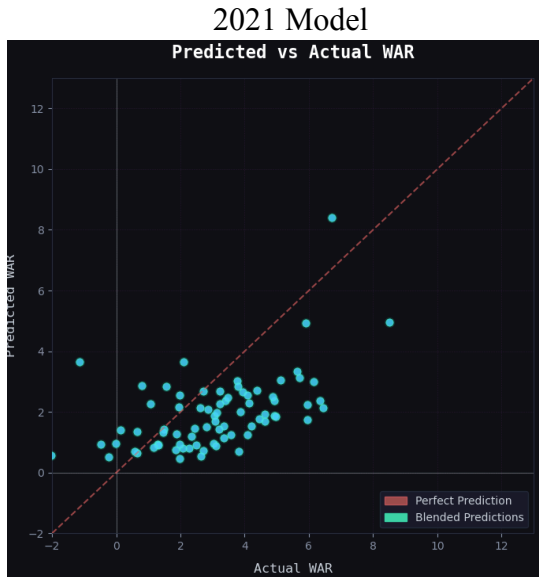


Figure 3.4

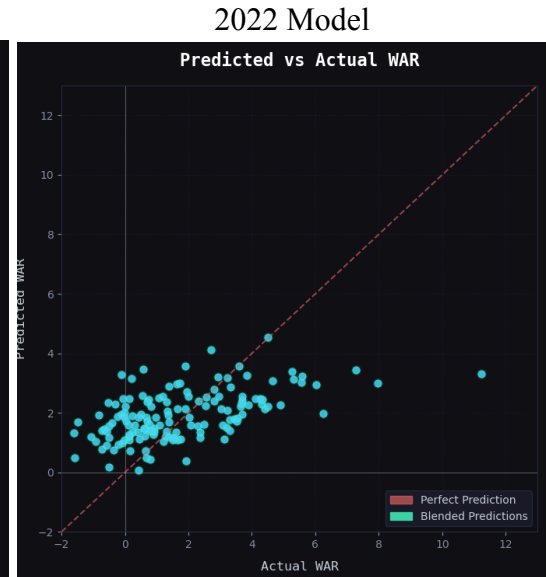


Figure 3.5

The most obvious reason for this is all three of these models take in the 2020 season which was greatly shortened to 60 games due to the pandemic. The biggest correlation to WAR is games played, the more games the more likely a player is to develop higher WAR, so playing less games will definitely hurt the prediction. This is just another fault of the model due to just taking straight counting stats and rate stats. Finally here is the 2020 model in which it predicts a full 162 game season where 60 games were played.

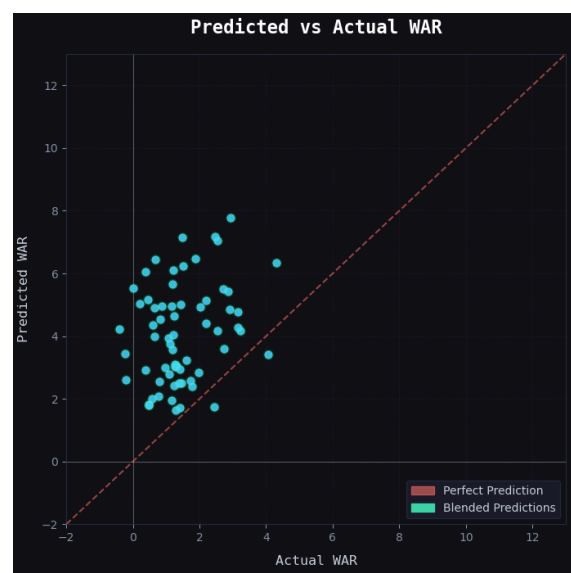


Figure 3.6

5.3 What I Learned

What I learned the most from this was how to construct a random forest model and how to boost the outliers of the model. My research into random forests was really interesting and from there I learned how gradient boosting worked XGBoosting. At first I wanted to add an XGBoost model to my project but then realized that was a bit too advanced with the time I had. To counteract this I decided that learning the basics of boosting and adding it to my random forests might get a better result. After a couple days of updating variables and researching both random forests and boosting I was able to get an algorithm that I thought was similar to the actual WAR values. I am hoping to learn more about gradient boosting in the future and will definitely be updating my algorithm to see better results.

References

"FanGraphs Baseball: Baseball Statistics and Analysis." *FanGraphs*, www.fangraphs.com/

"Lahman Baseball Database." *Society for American Baseball Research*,
sabr.org/lahman-database/.

"Ridge." *Scikit-Learn*,
scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html.

"Matplotlib: Visualization with Python." *Matplotlib*, matplotlib.org/stable/.

"Random Forest Algorithm in Machine Learning." *GeeksforGeeks*, 6 Apr. 2026,
www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/

Kavlakoglu, Eda. "What Is Random Forest?" *IBM*, www.ibm.com/think/topics/random-forest.

"Boosting in Machine Learning." *GeeksforGeeks*, 18 Feb. 2026,
www.geeksforgeeks.org/machine-learning/boosting-in-machine-learning/.

"What Is Boosting?" *IBM*, www.ibm.com/think/topics/boosting.

"Gradient Boosting in ML." *GeeksforGeeks*, 3 Dec. 2025,
www.geeksforgeeks.org/machine-learning/ml-gradient-boosting/.

Acknowledgments

I acknowledge that I used AI in helping me with designing my graphs and plots with matplotlib. This has allowed me to go further in depth and learn more about how plots work inside of python and how to design them. I also acknowledge the use of prior work in this class regarding my ridge regression model

Source Code

<https://github.com/jneeley3/Predicting-Major-League-Baseball-Future-Wins-Above-Replacement>